



Hexfields: Dominion

Handout zur Abschlusspräsentation am 03.06.2026

1) Statistiken über die Aufwändungen (1d = 24h; 1h = 60m)

a) Stunden pro Person und Hauptbeitrag

Person	Erfasste Zeit	Hauptbeitrag
Alexander Horst	2d 16h 36m	Match Seite
Marcel Steinle	2d 12h 30m	Design & Dokumente
Jona Lauber	2d 18h 5m	Auth, Dialogs, Trading

b) Stunden pro Workflow

Workflow	Erfasste Zeit
Requirements	1d
Analysis & Design	15h 30m
Implementation	6d 11h 10m
Test	8h 55m
Deployment	5h 20m
Configuration & Change Mgmt.	4h 30m
Project Management	9h 10m
Environment	5h 41m

c) Stunden pro Phase

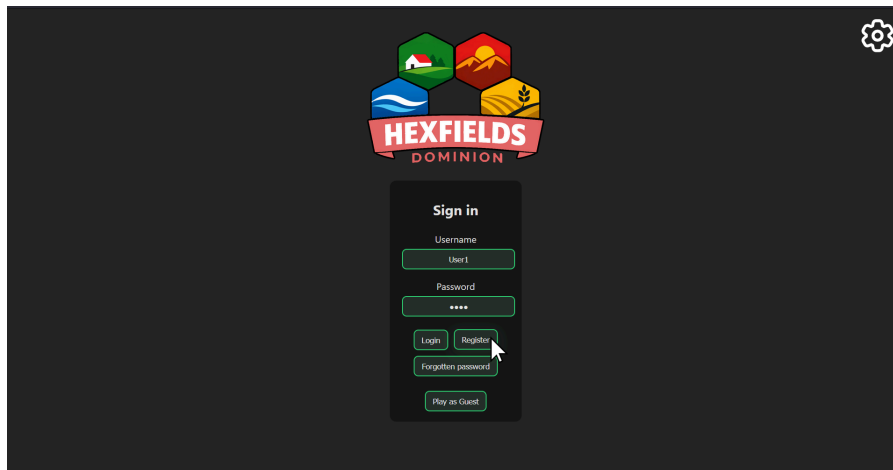
Phase	Erfasste Zeit
Inception	1d 10h 40m
Elaboration	15h 45m
Construction	7d 9h 18m
Transition	0m (Phase nicht gestartet)



2) Präsentations-Highlights

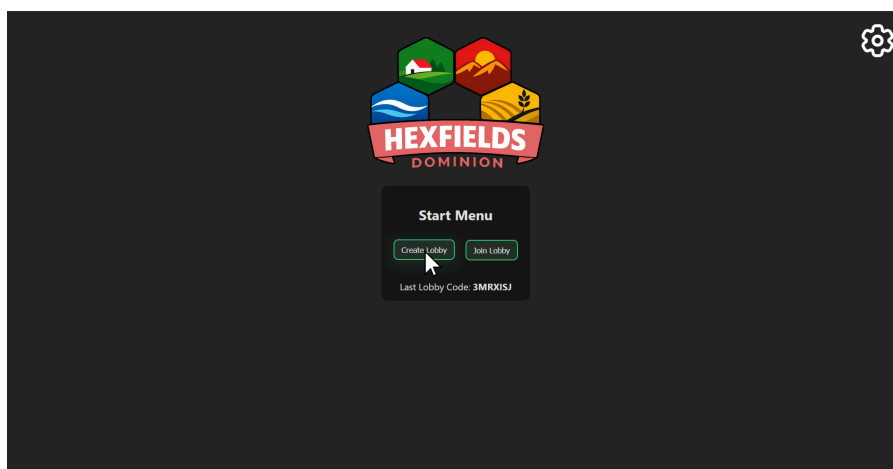
a. Login-Bildschirm

- ❖ Besuchen des Frontend über GitHub Pages
- ❖ Registrieren und Anmelden als Spieler
- ❖ Anmelden als Gast-Spieler
- ❖ Erklärung Vorteile von registrierten Nutzern (Konzept: Persistente Speicherung des Spiels)



b. Lobby-Beitritt auf Start-Bildschirm

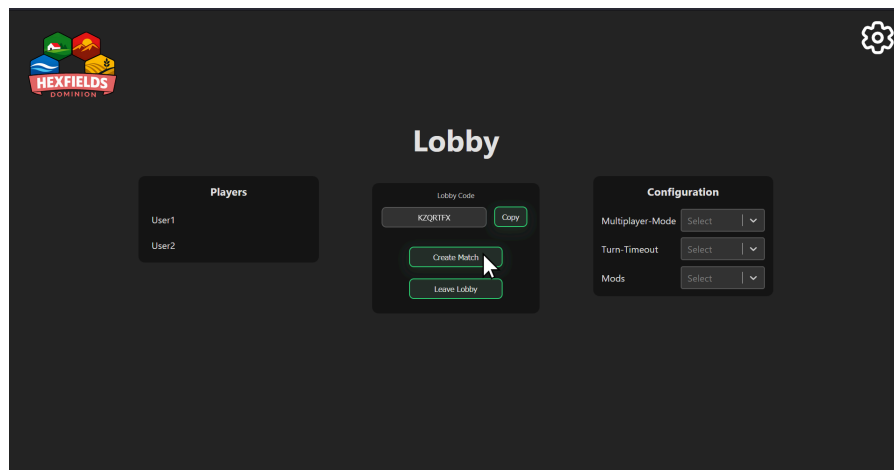
- ❖ Einstellen des Light- und Dark-Mode im Zahnrad-Menü
- ❖ Wiederholtes Abmelden und Anmelden durch das Zahnrad-Menü
- ❖ Erklärung von Dialogfenster und LocalStorage
- ❖ Erstellen einer Lobby (PC am Projektor)
- ❖ Beitreten einer Lobby (anderer Spieler)





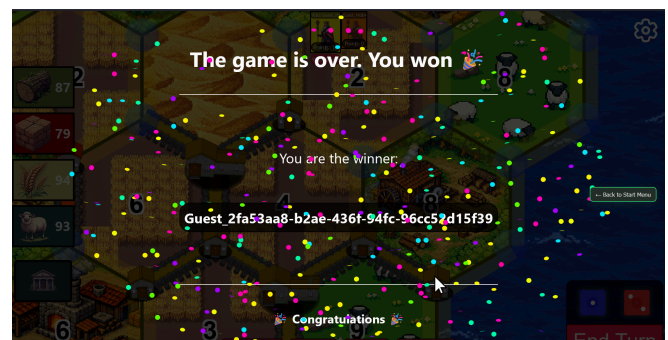
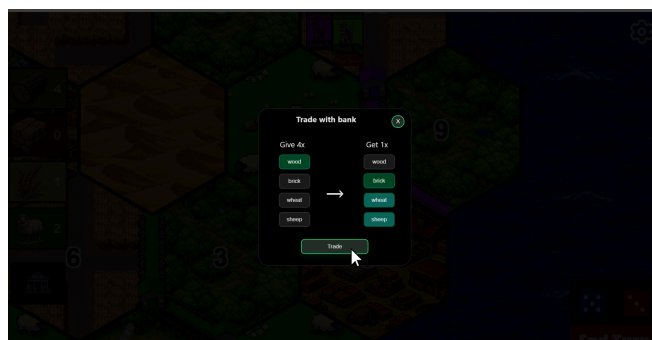
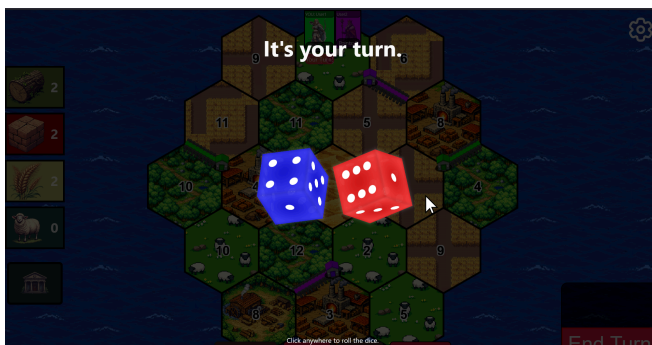
c. In der Lobby

- ❖ Live-Update der Spielerliste bei Beitritt und Verlassen
- ❖ Kopieren des Lobby-Code



d. Im Spiel

- ❖ Erläuterung der Spiel-GUI
- ❖ Kompletten Spielzyklus langsam durchgehen:
- ❖ Start-Bedingungen (Häuser und Ressourcen)
- ❖ Ein Spielzug (Würfeln, Bauen von Straßen und Häusern, Siegpunkte, Zug beenden, Handeln)
- ❖ Siegerehrung auf dem Projektor



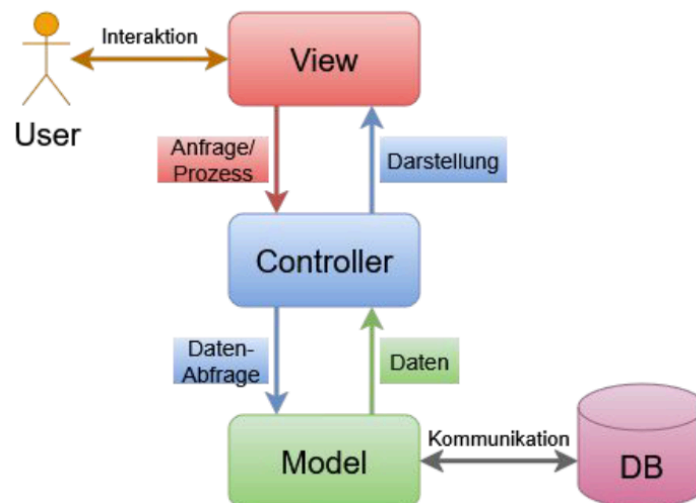


3) Projekt-Highlights

a. Architekturen

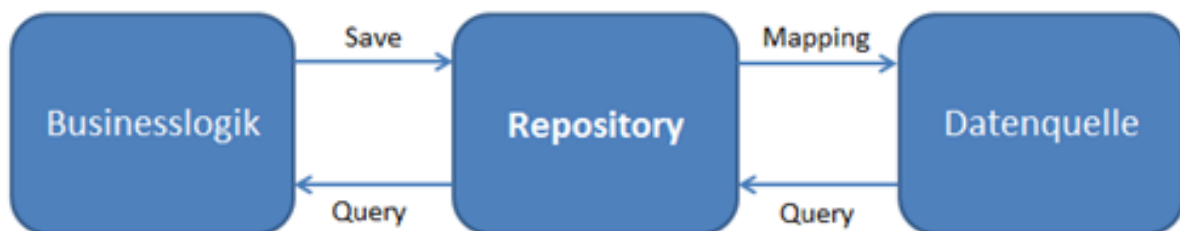
MVC-Modell

Die Datenbank, das Frontend und Backend arbeiten wie im typischen Model-View-Controller-Modell zusammen.



Repository-Pattern

Das Backend implementiert das Repository-Pattern. DB-Anfragen im Code laufen immer über ein Repository ab, wobei Java-Repository-Klassen als Schnittstellen zur Datenbank agieren. Java Objekte dagegen repräsentieren die einzelnen Tabellen.



Im Frontend haben wir auch das Repository-Pattern implementiert, um die Businesslogik von Datenaufnahme zu trennen. Über SSE sammeln wir alle Match-Daten und speichern diese in das Repository, welche mit Funktionen beliebig abgefragt werden können. React-Components können z.B. die benötigten Daten durch die Verwendung von selbst erstellten React-Hooks erhalten, die an das Repository abonniert sind, sodass React eine Änderung der Daten erkennen und einen Re-Render auslösen kann.



Code-Struktur

Das Frontend ist wegen React Component-basiert aufgebaut.

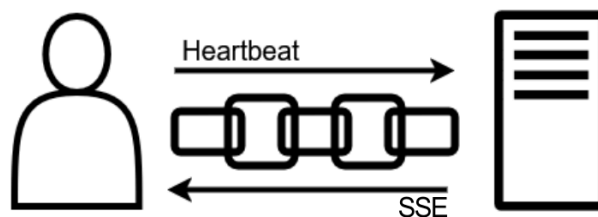
→ z.B. components/ → UI-Elemente.

Das Backend ist wegen Repository-Pattern Package-basiert aufgebaut.

→ z.B. account/ → Auth & Login.

SSE und Heartbeat - Verbindung von Backend zu Client

SSE (Server-Sent-Events) wurde gewählt, da es im Vergleich zu einem Websocket weniger Ressourcen verwendet und daher für diese Anwendung optimal ist (Anfragen werden in relativ geringer Rate gesendet). Um die Verbindung zum Backend aufrechtzuerhalten, wird ein Heartbeat-Mechanismus eingesetzt. Mit Token-Caching im Backend wird die Anmeldung beschleunigt.



Lobby- und Match-Organisation

Hexfields: Dominion spielt sich innerhalb von Matches ab, welche innerhalb einer Lobby erstellt wurden. Lobbies sind in deren Anzahl begrenzt, um DOS-Attacken zu vermeiden.

Jede Lobby und jedes Match muss einzigartig und identifizierbar sein:

- Ein Lobby-Code ist eine einfache Abfolge von 7 Zeichen, die geteilt werden kann, damit andere Spieler die Lobby betreten können. Das URL-Schema ist:

 hexfields-studio.github.io/HexfieldsDominion/lobby/NY0L8CA

- Ein Match-Code ist eine UUID. Diese ermöglicht Langzeitaufbewahrung und eindeutige Zuordnung, ohne dass eine Dopplung auftritt. Das URL-Schema ist:

 hexfields-studio.github.io/HexfieldsDominion/match/6bca49e2-557c-4523-b742-fc4e7f17adc0



b. Software (Tools / Plattform / Technik / Libraries)

Typ	Frontend	Backend
IDE	Visual Studio Code	IntelliJ IDEA
Package Manager	bun	gradle
Framework	React TypeScript	Java Spring Boot
Tools	eslint	eslint, Docker, SonarQube, MetricsTree
Plattform	GitHub Pages	Render
Wichtigste Bibliotheken	React (Router, DOM, Select), Sass, Konva, JWT-decode, js-confetti	Spring Boot (Web, Security, Data JPA), JWT, PostgreSQL, Lombok, bouncycastle
Sonstiges	Neon (Datenbank-Host), Jira (Projektmanagement)	

c. Datenbank Design

❖ Tabelle `_user`

- Speichert Nutzernamen und Passwort-/ Refresh-Token-Hashes
- Kommuniziert ausschließlich mit dem Backend

❖ Einzelner Host, erreichbar auf Neon-Service.

❖ Weitere Tabellen zur konzipierten Speicherung von Match-Daten wurden wegen Zeitdruck nicht implementiert.

d. Tests

❖ Frontend

- Ziel: 10-20% Abdeckung
- Fokus: Authentifizierung, Schnittstellen
- Aktuell: 0% Abdeckung
- Technologien: N/A (Keine Tests implementiert)

❖ Backend

- Ziel: ~80% Abdeckung
- Fokus: Lobby-/Spiellogik, Authentifizierung
- Aktuell: ~36% Abdeckung
- Technologien: Mockito, JUnit



e. Metriken

- ❖ Frontend: N/A (Keine Metriken implementiert)
- ❖ Backend:
 - Erhebung durch Jacoco und SonarQube in der CI/CD-Pipeline
 - Manuelle Erhebung durch MetricsTree-Plugin für IntelliJ

Metrik	Ergebnis	Evaluierung
Halstead-Metriken	Difficulty: ~1.600 Effort: ~2.200.000 Errors: ~14 Length: ~4.400 Vocabulary: ~1.900 Volume: ~25.600	Hohe Difficulty, Effort und Vocabulary; Erschwerte Wartbarkeit + höhere Fehler-Wskt. durch Umfang des Codes. Im Vergleich zu Blog-Post 16 deutlicher Anstieg aller Werte (teilweise 3-4-fach)
Depth-Of-Inheritance (DIT)	1: ~50% 2: ~30% 3, 4: ges. 10 Fälle 5: ~15% 6: 1 Fall	DIT 1-4 ist akzeptabel. Ausreißer mit DIT ≥ 5 eher problematisch, DIT6-Klasse könnte Probleme bzgl. der Wartbarkeit bereiten Im Vergleich zu Blog-Post 16 wurde Lobby-Management implementiert, wodurch DIT5 und DIT6 erreicht wurden.
McCabe'sche Zyklomatische Komplexität (CC)	1: ~80% 2: ~5% 3: ~5% 4-10: ~5% 16: 1 Fall	Insgesamt gute Komplexität, fast gesamt CC < 10, wenige logische Abzweigungen. Es sind keine Änderungen notwendig, da alle Werte <20 sind. Sicherheitsrelevante Methoden sind ≤7. Fall mit 16 ist eine generierte Klasse (vmtl. =False-Positive).



f. CI/CD

- ❖ Alle Pipelines haben Build- und Test-Stages und werden auf externen Services autonom deployed.
- ❖ Da die Anwendung direkt im Browser des Endgeräts geöffnet wird, muss kein Client installiert oder aktualisiert werden.
- ❖ Die CI/CD-Pipelines werden bei Pull-Request (Code-Prüfung) und bei Push auf Main ausgelöst.
- ❖ Frontend:
 - Build-Stage: Checkout, Setup, Dependencies, TypeScript-Build, Upload Artifact
 - Test-Stage: Checkout, Setup, Dependencies, Download Artifact, Type-Check, Tests (nicht implementiert)
 - Deploy-Stage: Checkout, Set up GitHub Pages, Upload Artifact to Pages and Deploy
- ❖ Backend:
 - Build-Stage: Checkout, Setup, Gradle Permission, Gradle-Build, Upload Artifact
 - Test-Stage: Checkout, Setup, Gradle-Test (mit SonarQube und Jacoco), Upload Coverage Report
 - Deployment: Automatischer Pull, Container-Build und Deploy auf Render, nicht von Pipeline beeinflusst (CronJob, tägliche Aktualisierung)